

```

dubot@CSIT345:~/Downloads$ gcc -o dp_synch dp_synch.c
dubot@CSIT345:~/Downloads$ ./dp_synch
Philosopher 0 is thinking...
Philosopher 3 is thinking...
Philosopher 4 is thinking...
Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking...
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking...
Philosopher 2 is eating.
Philosopher 0 is eating.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking...
Philosopher 4 is eating.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 1 is eating.
Philosopher 2 is thinking...
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking...
Philosopher 3 is eating.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking...
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking...
Philosopher 0 is eating.
Philosopher 3 is eating.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking...
Philosopher 1 is eating.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking...
Philosopher 4 is eating.
^C
dubot@CSIT345:~/Downloads$ █

```

dc.p

When running dp.c without synchronization, philosophers attempt to pick up chopsticks at the same time. This leads to a situation where each philosopher may hold one chopstick and wait indefinitely for the other, causing a deadlock. The output appears inconsistent and unsynchronized, with multiple philosophers trying to eat at the same time, violating proper resource sharing.

Output differences

After implementing synchronization, the output becomes orderly and controlled. Philosophers only eat when both neighboring chopsticks are available, and at most two philosophers eat at the same time. Deadlocks are prevented because philosophers wait when resources are unavailable, and condition variables ensure proper signaling between threads. Overall, synchronization ensures safe and efficient resource sharing compared to the unsynchronized version.